# SpeakPrep Dev Playbook

Abhiyan Sainju

April 2026

## SpeakPrep — Document 5: Full Development Playbook

**Exact Prompts · Git Strategy · CI/CD · Mockups · Agent Assignments**

**Version 1.0 | Abhiyan Sainju | April 2026**

---

This document tells you exactly what to do, in what order, with what commands and prompts, before and during every phase of building SpeakPrep. Read Part 1 fully before touching anything. Every other part is a reference — open it when you reach that phase.

---

## PART 0: EVERYTHING TO SET UP BEFORE WRITING CODE

Do this in one sitting. It takes 3–4 hours. Do it once, correctly.

---

### Step 0.1 — Accounts (30 min)

Create these accounts if you don't have them. Save every API key in a password manager (Bitwarden, 1Password) immediately.

| Service | URL | Purpose |
|---|---|---|
| GitHub | github.com | Code, CI/CD |
| Deepgram | deepgram.com | ASR ($200 credit) |
| Groq | console.groq.com | LLM (free) |
| Supabase | supabase.com | PostgreSQL + Auth |
| Oracle Cloud | oracle.com/cloud/free | Compute (ARM, free) |
| Cloudflare | cloudflare.com | DNS + Tunnel (free) |
| Sentry | sentry.io | Error tracking (free) |
| PostHog | posthog.com | Analytics (free) |
| New Relic | newrelic.com | APM (free) |
| UptimeRobot | uptimerobot.com | Uptime alerts (free) |
| Figma | figma.com | UI mockups (free) |

After creating each account, immediately grab the API key and store it. You'll need them all in Step 0.4.

---

**Step 0.2 — Local Dev Tools (30 min)**

Install everything before writing a single line of code. Missing tools mid-build breaks flow.

```
# 1. Python version management
curl https://pyenv.run | bash
# Add to ~/.bashrc or ~/.zshrc:
# export PYENV_ROOT="$HOME/.pyenv"
# export PATH="$PYENV_ROOT/bin:$PATH"
# eval "$(pyenv init -)"
source ~/.bashrc
pyenv install 3.12.3
pyenv global 3.12.3
python --version    # → Python 3.12.3

# 2. Poetry (dependency management)
curl -sSL https://install.python-poetry.org | python3 -
poetry --version    # → Poetry 1.8.x

# 3. Node (for frontend)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 20
nvm use 20
node --version     # → v20.x.x

# 4. Docker
# Mac: download Docker Desktop from docker.com/get-started
# Linux:
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
# Log out and back in
docker --version
docker compose version

# 5. CLI tools
pip install httpie       # Better curl for API testing
npm install -g wscat     # WebSocket testing
brew install jq          # JSON processing (Mac)
# or: sudo apt-get install jq (Linux)

# 6. Cloudflared (for local HTTPS tunnel during dev)
# Mac: brew install cloudflare/cloudflare/cloudflared
```

```
# Linux:
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudfla
  -o /usr/local/bin/cloudflared
chmod +x /usr/local/bin/cloudflared

# 7. Git configuration (important — sets your identity in commits)
git config --global user.name "Abhiyan Sainju"
git config --global user.email "your@email.com"
git config --global core.editor "code --wait"   # VSCode as git editor
git config --global init.defaultBranch main

# 8. GitHub CLI
# Mac: brew install gh
# Linux: see cli.github.com/manual/installation
gh auth login   # Authenticate

# 9. VSCode extensions to install
code --install-extension ms-python.python
code --install-extension ms-python.pylance
code --install-extension ms-azuretools.vscode-docker
code --install-extension eamodio.gitlens
code --install-extension humao.rest-client
code --install-extension ms-vscode.live-server
```

---

**Step 0.3 — Repository Setup (45 min)**

This is the most important setup step. Every decision here affects every day of development.

```
# Create repo on GitHub first (via web or CLI)
gh repo create speakprep --private --description "Real-time voice AI mock interview
gh repo clone speakprep && cd speakprep

# Create the full project structure in one command
mkdir -p \
  backend/app/{api,services,models,audio,utils} \
  backend/tests/{unit,integration} \
  backend/alembic/versions \
  frontend/src/{components,pages,hooks,store,utils} \
  docs/adr \
  scripts \
  .github/workflows

# Create all essential files
touch backend/app/__init__.py
touch backend/app/api/__init__.py
touch backend/app/services/__init__.py
```

```
touch backend/app/models/__init__.py
touch backend/app/audio/__init__.py
touch backend/app/utils/__init__.py
touch backend/tests/__init__.py
touch backend/tests/unit/__init__.py
touch backend/tests/integration/__init__.py

# Initialize Poetry for backend
cd backend
poetry init --name speakprep-backend --python "^3.12" --no-interaction
cd ..
```

**Create .gitignore — critical:**

```
# .gitignore
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
.venv/
venv/
env/
*.egg-info/
.eggs/
dist/
build/
*.pyc

# Environment & Secrets
.env
.env.*
!.env.example
*.key
secrets/

# Node
node_modules/
.next/
dist/
build/
*.tsbuildinfo

# IDE
.vscode/settings.json
.idea/
*.swp
*.swo
```

```
.DS_Store
Thumbs.db

# Docker
*.log

# AI/ML Models (never commit model weights)
*.bin
*.onnx
*.pt
*.ckpt
*.safetensors
models/

# Pytest cache
.pytest_cache/
.coverage
htmlcov/
coverage.xml

# Mypy
.mypy_cache/

# Compiled LaTeX (keep .tex source, not .pdf)
*.aux
*.fls
*.fdb_latexmk
*.out
*.toc
*.synctex.gz
```

**Create CONTEXT.md — your agent shared memory:**

```
# CONTEXT.md — Agent Shared Memory
# Update this every time you finish a task or switch agents

## Project: SpeakPrep — Real-time Voice AI Mock Interview Coach
## Author: Abhiyan Sainju
## Started: April 2026

## Current Phase: 0 — Environment Setup
## Current Task: Initial repository setup

## Architecture Summary
- Backend: Python 3.12, FastAPI, asyncio, WebSockets
- ASR: Deepgram Nova-3 (primary), Faster-Whisper (fallback)
- LLM: Groq / Llama 3.3 70B (primary), Cerebras (fallback)
- TTS: Kokoro TTS self-hosted via Docker
- DB: Supabase PostgreSQL (Phase 1-2), self-hosted later
```

```
- Cache: Valkey (Redis-compatible)
- Infra: Oracle Cloud ARM, Docker Compose, Cloudflare Tunnel

## Docs Reference
- PRD: /docs/doc1-prd-product.md
- Architecture: /docs/doc2-system-design-architecture.md
- Build Guide: /docs/doc3-build-guide.md
- Portfolio: /docs/doc4-interview-portfolio.md

## What's Working
- [ ] Nothing yet — day 0

## What's In Progress
- [x] Repository setup

## What's Blocked
- Nothing

## Key Files
- Entry point: backend/app/main.py (not created yet)
- DB models: backend/app/models/ (not created yet)
- WebSocket handler: backend/app/api/ws.py (not created yet)

## DO NOT TOUCH
- Nothing locked yet

## Last Updated
- Date: [today]
- By: Abhiyan (setup session)
```

**Copy your docs into the repo:**

```
cp /path/to/doc1-prd-product.md docs/
cp /path/to/doc2-system-design-architecture.md docs/
cp /path/to/doc3-build-guide-curriculum.md docs/
cp /path/to/doc4-interview-portfolio.md docs/
cp /path/to/doc5-dev-playbook.md docs/
```

**First commit — the foundation:**

```
git add .
git commit -m "chore: initial repository structure and documentation

- Add full directory structure for backend and frontend
- Add .gitignore with Python, Node, secrets, and model exclusions
- Add CONTEXT.md for agent shared memory
- Copy all 5 SpeakPrep blueprint docs into /docs
- No code yet — this is the scaffold"
```

```
git push origin main
```

---

**Step 0.4 — Environment Variables (15 min)**

```
# Create .env.example (commit this — no real values)
cat > .env.example << 'EOF'
# API Keys
DEEPGRAM_API_KEY=your_deepgram_key_here
GROQ_API_KEY=your_groq_key_here
CEREBRAS_API_KEY=your_cerebras_key_here

# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your_anon_key_here
DATABASE_URL=postgresql://user:password@host:5432/speakprep

# Auth
JWT_SECRET_KEY=generate_with_openssl_rand_hex_32

# Service URLs (Docker internal)
VALKEY_URL=redis://valkey:6379
KOKORO_URL=http://kokoro:8880
WHISPER_URL=http://whisper:9000

# Monitoring
SENTRY_DSN=https://your_sentry_dsn
NEW_RELIC_LICENSE_KEY=your_newrelic_key
POSTHOG_API_KEY=your_posthog_key

# Environment
ENVIRONMENT=development    # development | production
LOG_LEVEL=INFO
EOF

# Create .env with your real values (NEVER commit this)
cp .env.example .env
# Now fill in .env with your real API keys

git add .env.example
git commit -m "chore: add .env.example with all required environment variables"
```

---

**Step 0.5 — Pre-commit Hooks (15 min)**

These run automatically before every commit. They catch secrets, bad formatting, and type errors before they hit GitHub.

```
pip install pre-commit gitleaks
cd backend && poetry add --group dev ruff mypy pytest pytest-asyncio pytest-cov

# Create .pre-commit-config.yaml
cat > .pre-commit-config.yaml << 'EOF'
repos:
  # Prevent secrets from being committed
  - repo: https://github.com/zricethezav/gitleaks
    rev: v8.18.2
    hooks:
      - id: gitleaks

  # Python formatting and linting
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.2
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  # General
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-json
      - id: check-merge-conflict
      - id: check-added-large-files
        args: ['--maxkb=1000']   # Block files over 1MB
      - id: no-commit-to-branch
        args: ['--branch', 'main']   # Can't commit directly to main
EOF

pre-commit install
git add .pre-commit-config.yaml
git commit -m "chore: add pre-commit hooks for secrets, linting, formatting"
```

---

**Step 0.6 — GitHub Actions CI/CD (45 min)**

Set this up before writing any application code. Having CI from day one means every commit is validated.

```
mkdir -p .github/workflows
```

**Create .github/workflows/ci.yml:**

```yaml
name: CI

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend

    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: speakprep_test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports: ["5432:5432"]

      valkey:
        image: valkey/valkey:8-alpine
        options: >-
          --health-cmd "valkey-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports: ["6379:6379"]

    steps:
      - uses: actions/checkout@v4
```

```
- name: Set up Python 3.12
  uses: actions/setup-python@v5
  with:
    python-version: "3.12"

- name: Install Poetry
  uses: snok/install-poetry@v1
  with:
    version: 1.8.3
    virtualenvs-in-project: true

- name: Cache dependencies
  uses: actions/cache@v4
  with:
    path: backend/.venv
    key: venv-${{ hashFiles('backend/poetry.lock') }}

- name: Install dependencies
  run: poetry install --with dev

- name: Lint (ruff)
  run: poetry run ruff check .

- name: Format check (ruff format)
  run: poetry run ruff format --check .

- name: Type check (mypy)
  run: poetry run mypy app/ --ignore-missing-imports
  continue-on-error: true   # Don't block on type errors in early phases

- name: Run tests
  env:
    DATABASE_URL: postgresql://test:test@localhost:5432/speakprep_test
    VALKEY_URL: redis://localhost:6379
    JWT_SECRET_KEY: test-secret-key-min-32-characters-long
    GROQ_API_KEY: test-key
    DEEPGRAM_API_KEY: test-key
    ENVIRONMENT: test
  run: |
    poetry run pytest tests/ \
      -v \
      --cov=app \
      --cov-report=xml \
      --cov-report=term-missing \
      -x   # Stop on first failure
  continue-on-error: true   # Tests may not exist yet in Phase 0

- name: Upload coverage
```

```
        uses: codecov/codecov-action@v4
        if: always()
        with:
          file: backend/coverage.xml
        continue-on-error: true
```

**Create .github/workflows/deploy.yml:**

```
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push backend image
        uses: docker/build-push-action@v5
        with:
          context: ./backend
          platforms: linux/arm64
          push: true
          tags: ghcr.io/${{ github.repository }}/backend:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Deploy to Oracle ARM
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.ORACLE_HOST }}
          username: ubuntu
          key: ${{ secrets.ORACLE_SSH_KEY }}
          script: |
```

```
cd ~/speakprep
git pull origin main
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d --force-recreate app
docker compose -f docker-compose.prod.yml exec -T app \
  alembic upgrade head
sleep 15
curl -f http://localhost:8000/api/health || exit 1
docker image prune -f
echo "Deploy successful"
```

**Add GitHub Secrets:**

```
# Go to: github.com/abhiyansainju/speakprep → Settings → Secrets →
Actions
# Add these secrets (values from your accounts):
ORACLE_HOST          # Your Oracle VM IP
ORACLE_SSH_KEY       # Your private SSH key content
```

```
git add .github/
git commit -m "ci: add GitHub Actions CI and deploy workflows

- CI: lint (ruff), format check, type check (mypy), pytest
- CD: build ARM64 Docker image, deploy to Oracle ARM
- Services: postgres and valkey for integration tests
- Coverage upload to Codecov"

git push origin main
```

---

**Step 0.7 — Branch Strategy Setup (5 min)**

```
# Create develop branch — all features merge here first
git checkout -b develop
git push origin develop

# Protect main branch on GitHub:
# Settings → Branches → Add rule
# Branch name pattern: main
# ☑ Require pull request reviews before merging
# ☑ Require status checks to pass (CI)
# ☑ Include administrators

# Protect develop:
# Settings → Branches → Add rule
# Branch name pattern: develop
# ☑ Require status checks to pass (CI)
```

---

**Step 0.8 — UI/UX Mockup Timing Decision**

**Do mockups BEFORE building the frontend (Phase 5), not at the start.**

Here's why this ordering matters:

```
Phase 0–4:  No UI needed. You're building the voice pipeline, API, and infrastructu
            Making Figma mockups now is a waste — you don't know what's feasible

Phase 5:    You're building the actual React frontend. Do mockups here, one week be
            By now you know: what data you have, what latency you achieved,
            what the real voice interaction feels like. Your mockups will be accur
```

**What to do NOW instead of full mockups:**

Sketch a rough user flow on paper (10 minutes). Just boxes and arrows:

Landing → Sign Up → Resume Upload → Session Setup → Voice Session → Report → Dashboard

That's enough to guide the API design. The detailed Figma mockups come in Phase 5.

**When you reach Phase 5, do this user flow research:** 1. Ask 5 friends to describe their dream interview practice tool (10-min call each) 2. Record what they say about existing tools (what's annoying, what's missing) 3. Build one Figma flow for the Voice Session page — that's the core experience 4. Test it with 2 users before writing frontend code

---

**Step 0.9 — Final Pre-Code Checklist**

```
☐
 All accounts created, API keys saved in password manager☐
 Local dev tools installed and verified (python 3.12, docker, node, wscat)☐
 Git repo created with proper structure☐
 .gitignore in place☐
 CONTEXT.md created☐
 .env.example committed, .env filled with real keys☐
 Pre-commit hooks installed☐
 GitHub Actions CI workflow running (push to main to verify)☐
 develop branch created☐
 Branch protections enabled on GitHub☐
 User flow sketch done (paper)
```

If any box is unchecked, do not proceed.

---

## PART 1: GIT DISCIPLINE — GRANULAR TRACKING

This is how you track every single change so you can revert anything.

---

### The Golden Rule: Commit Every Logical Unit

A "logical unit" is the smallest change that makes sense on its own. Examples:

```
✅
 Add one function✅
 Add one test✅
 Fix one bug✅
 Add one import✅
 Change one config value✅
 Rename one variable across the codebase✅
 Add one environment variable to .env.example✅
 Update one line in CONTEXT.md✅

 "Add all the audio stuff" (too big)❌
 "Work in progress" (meaningless)❌
 "Fix" (what fix?)❌
 Committing 500 lines at once
```

### How to Stage Changes Granularly

```
# NEVER do this:
git add .        # Stages everything blindly
git add -A       # Same problem

# ALWAYS do this instead:
git add -p       # Interactive patch — review every change before staging
```

`git add -p` shows you each change and asks: Stage this hunk [y,n,q,a,d,/,s,e,?]? - y — yes, stage this change - n — no, skip it - s — split into smaller chunks - e — manually edit what to stage

**Example of granular commit workflow:**

```
# You just added VAD detection + a helper function + updated CONTEXT.md
git add -p

# Review hunk 1: new import line
# → y (stage it)

# Review hunk 2: VADRecorder class (40 lines)
# → y (stage it)

# Review hunk 3: CONTEXT.md update
```

```
# → n (save for separate commit)

git commit -m "feat(audio): add VADRecorder class with WebRTC VAD

- VADRecorder collects audio frames until sustained silence detected
- Configurable: silence_threshold_ms, min_speech_ms, vad_aggressiveness
- Returns numpy int16 array of speech frames only
- Based on webrtcvad library, mode=2 (medium aggressiveness)"

# Now stage and commit the CONTEXT.md separately
git add CONTEXT.md
git commit -m "docs: update CONTEXT.md - VADRecorder complete, Phase 1 in progress"
```

**Commit Message Format (Strict)**

```
type(scope): short description (max 72 chars)

Optional longer description explaining WHY, not just what.
What was the problem? What approach did you take? Any tradeoffs?

Refs: #issue-number (if applicable)
```

**Types:**

```
feat —      New feature or function
fix —       Bug fix
test —      Adding or modifying tests
docs —      Documentation only (CONTEXT.md, README, comments)
refactor —  Code restructuring (no behavior change)
perf —      Performance improvement
chore —     Build, config, deps, CI changes
style —     Formatting only (shouldn't happen — pre-commit handles this)
revert —    Reverting a previous commit
wip —       Work in progress (only on feature branches, never merge to develop)
```

**Scopes (for SpeakPrep):**

```
audio —     Audio capture, PCM, frames
vad —       Voice activity detection
asr —       Speech to text (Deepgram, Whisper)
llm —       LLM integration (Groq, fallbacks)
tts —       Text to speech (Kokoro)
ws —        WebSocket handler
api —       REST API endpoints
auth —      Authentication, JWT
db —        Database models, migrations
cache —     Valkey/Redis
pipeline —  Full voice pipeline integration
scoring —   AI scoring system
```

```
resume —      Resume parsing
questions —   Question bank and ELO
dashboard —   Progress tracking UI
ci —          CI/CD configuration
infra —       Docker, deployment
```

**How to Revert — Every Scenario**

```
# Scenario 1: Undo last commit but keep the changes
git reset HEAD~1
# Your changes are now unstaged. Recommit when ready.

# Scenario 2: Completely undo last commit (discard changes)
git reset --hard HEAD~1
# ⚠ Destructive — changes gone forever

# Scenario 3: Revert a specific commit (creates a new "undo" commit)
git revert abc1234
# Safe — doesn't rewrite history. Use this on shared branches.

# Scenario 4: Undo a specific file to last commit state
git checkout HEAD -- backend/app/audio/vad_recorder.py

# Scenario 5: Undo a specific file to a specific commit
git checkout abc1234 -- backend/app/audio/vad_recorder.py

# Scenario 6: See what changed in a commit
git show abc1234

# Scenario 7: Find when a bug was introduced (bisect)
git bisect start
git bisect bad                  # Current state is broken
git bisect good abc1234         # This commit was good
# Git checks out middle commits — test each one
git bisect good   # or: git bisect bad
# Repeat until git finds the first bad commit
git bisect reset  # Return to HEAD

# Scenario 8: Undo a change to one specific function (interactive rebase)
git log --oneline -10           # Find the commit
git rebase -i HEAD~5            # Interactive rebase last 5 commits
# In editor: change "pick" to "edit" for the commit to modify
# Git stops at that commit — make changes, then:
git add -p && git commit --amend
git rebase --continue

# Scenario 9: Stash work in progress before switching tasks
git stash push -m "wip: half-done sentence splitter"
```

```
git stash list                  # See all stashes
git stash pop                   # Apply most recent stash
git stash apply stash@{2}       # Apply specific stash
```

### Branch Workflow Per Feature

```
# Start every new feature from develop (not main)
git checkout develop
git pull origin develop
git checkout -b feature/vad-recorder

# Work on it... commit often (see above)
# When ready to merge:
git checkout develop
git pull origin develop
git merge feature/vad-recorder
# Or: open a Pull Request on GitHub for code review history

git push origin develop

# Delete feature branch after merge
git branch -d feature/vad-recorder
git push origin --delete feature/vad-recorder
```

### Tagging Milestones

```
# Tag each phase completion — instant rollback point
git tag -a v0.1.0-phase0 -m "Phase 0 complete: environment setup"
git tag -a v0.2.0-phase1 -m "Phase 1 complete: local voice pipeline working"
git tag -a v0.3.0-phase2 -m "Phase 2 complete: WebSocket streaming working"
git push origin --tags
```

---

## PART 2: AGENT ASSIGNMENTS + EXACT PROMPTS

---

### The Agent Handoff Protocol (Do This Every Time)

Before switching from one agent to another:

```
# 1. Commit your current state
git add -p
git commit -m "wip: [what you were doing] - switching agents"

# 2. Update CONTEXT.md (2 minutes max)
# Open CONTEXT.md, update:
```

```
# - Current Task
# - What's Working (add completed item)
# - What's In Progress (update)
# - What's Blocked (if anything)

git add CONTEXT.md
git commit -m "docs: update CONTEXT.md before agent switch"

# 3. Push
git push origin [current-branch]
```

Then give the new agent the handoff prompt (see below for each agent).

---

**Cursor IDE — Daily Driver**

**When to use Cursor:** - Editing specific files - Adding one function to an existing file - Debugging a specific error - Quick refactors within a file - Understanding code you're reading

**Cursor Opening Prompt (use at start of every session):**

```
Read CONTEXT.md first. Then read the files listed in "Key Files" section.

I'm building SpeakPrep - a real-time voice AI mock interview coach.
Full architecture in /docs/doc2-system-design-architecture.md
Current phase and task are in CONTEXT.md.

Rules:
- Keep all code in files (never inline in chat)
- Conventional commits for every change: feat/fix/test/docs/refactor(scope): descri
- Ask me before creating new files I haven't mentioned
- If you're unsure about architecture, check /docs/ before guessing
- Run existing tests before and after changes: cd backend && poetry run pytest test

What's the current task in CONTEXT.md?
```

**Cursor Prompt for Adding a Function:**

```
Add a function `[function_name]` to [file path].

Requirements:
- [what it should do]
- [inputs and types]
- [outputs and types]
- [edge cases to handle]

Follow the existing code style in that file.
Add a docstring.
```

```
Add type hints (Python 3.12 style, use `X | None` not `Optional[X]`).
Do not modify any other function.
After writing it, write one pytest test for it in tests/unit/test_[module].py.
```

**Cursor Prompt for Fixing a Bug:**

```
I have a bug. Here's exactly what's happening:
[paste error message or describe behavior]

File: [path/to/file.py]
Function: [function_name]
Line (approximately): [line number]

What I expected: [expected behavior]
What's actually happening: [actual behavior]

Do NOT refactor anything else.
Fix only this specific issue.
After fixing, explain in one sentence why it was wrong.
```

**Cursor Prompt for Understanding Code:**

```
Explain what this code does, without changing it:
[paste code block]

Specifically:
1. What is the main purpose?
2. What does each parameter do?
3. Are there any edge cases or gotchas?
4. Is there anything that could go wrong?
```

---

**Claude Code — Architect + New Modules**

**When to use Claude Code:** - Building a new module from scratch (e.g., the full VAD recorder, the scoring service) - Cross-file refactoring (renaming something used everywhere) - Debugging complex async issues - Writing a full test suite for a service - “Build this whole thing” tasks

**Claude Code Opening Prompt:**

```
Read these files in order:
1. CONTEXT.md
2. /docs/doc2-system-design-architecture.md (the section relevant to current task)
3. Any existing files in the module I'm about to work on

Project: SpeakPrep - real-time voice AI mock interview coach
Stack: Python 3.12, FastAPI, asyncio, WebSockets, Groq, Deepgram, Kokoro TTS
Current phase: [X] — [phase name]
```

```
Current task: [specific task]

Architecture decisions are in /docs/adr/ — respect them, don't override them.
All code must have type hints, docstrings, and follow existing style.
Every new module needs a corresponding test file.
Commit each logical unit separately with conventional commit format.
```

**Claude Code Prompt for a New Service:**

```
Build the [ServiceName] service at backend/app/services/[service_name].py

Purpose: [what it does in one sentence]

Interface (these are the public methods it must expose):
- [method_name(param: type, param2: type) -> return_type]: description
- [method_name(param: type) -> return_type]: description

Implementation requirements:
- [specific requirement 1]
- [specific requirement 2]
- [specific requirement 3]

Do NOT use:
- [thing to avoid and why]

Dependencies already available (check pyproject.toml):
- [dep1, dep2]

After implementing:
1. Write unit tests at backend/tests/unit/test_[service_name].py
2. Test all public methods with at least: happy path, edge case, error case
3. Update CONTEXT.md to show this service is complete
4. Commit each file separately with proper conventional commits
```

**Claude Code Prompt for Cross-File Refactor:**

```
Refactor: [describe the change]

Files affected:
- [file1.py]: [how it changes]
- [file2.py]: [how it changes]

Why: [reason for refactor]

Rules:
- Do NOT change behavior, only structure
- All existing tests must still pass after refactor
- Run `poetry run pytest tests/` before and after, confirm same results
- Commit the refactor as one commit: refactor(scope): description
```

```
- Do not add new features during this refactor
```

---

**Codex CLI — Quick Scripts and Utilities**

**When to use Codex:** - One-off data scripts - Database seed scripts - Quick bash utilities - Generating boilerplate (Dockerfile, config files) - Converting data formats

**Codex Prompt Template:**

```
Write a Python script at scripts/[script_name].py that:
[describe what it does in bullet points]

Input: [what it takes as input]
Output: [what it produces]

Make it runnable as: python scripts/[script_name].py [args]
Add --help with argparse.
Add error handling for [specific error cases].
No external dependencies beyond: [list deps]
```

---

**Antigravity — Browser + Research Agent**

**When to use Antigravity:** - Testing the web UI during development - Researching competitor features - Checking API documentation - UI/UX validation (does the flow make sense?) - Web scraping for question bank content

**Antigravity Prompt for UI Testing:**

```
Test the SpeakPrep web app at http://localhost:5173

Test this user flow:
1. [step 1]
2. [step 2]
3. [step 3]

For each step, report:
- Did it work as expected?
- Any visual issues (misaligned, overflow, wrong color)?
- Console errors?
- Network errors in DevTools?
- How long did it take?

Take a screenshot after each step.
```

**Antigravity Prompt for Research:**

```
Research: [specific question about competitor or technology]

Go to [specific URLs or search for specific terms].
Report:
- Key findings
- Any relevant pricing or limitations
- Links to documentation

Do not sign up for anything or enter personal information.
```

---

## PART 3: PHASE-BY-PHASE AGENT PROMPTS

---

### PHASE 0: Environment + Foundations (Week 1-3)

**Git branch:** feature/phase0-foundations

**Task 0.1: Install Python Dependencies** **Agent: Cursor or Claude Code**

**cd** backend

**Prompt:**

```
Set up the Python project dependencies for SpeakPrep backend.

Add these to pyproject.toml using poetry add:

Core:
- fastapi[standard]>=0.111
- uvicorn[standard]>=0.29
- uvloop>=0.19
- websockets>=12.0
- python-jose[cryptography]>=3.3      # JWT
- passlib[bcrypt]>=1.7                # Password hashing
- httpx>=0.27                         # Async HTTP client
- pydantic>=2.7                       # Data validation
- pydantic-settings>=2.2              # Settings management
- sqlalchemy[asyncio]>=2.0            # ORM
- asyncpg>=0.29                       # PostgreSQL async driver
- alembic>=1.13                       # DB migrations
- structlog>=24.1                     # Structured logging

Audio/AI:
- faster-whisper>=1.0                 # Local ASR fallback
- openai-whisper>=20231117            # For Whisper utilities
- sounddevice>=0.4                    # Local audio (Phase 1 only)
```

```
- soundfile>=0.12                    # Audio file I/O (Phase 1 only)
- webrtcvad-wheels>=2.0              # Voice Activity Detection
- numpy>=1.26                        # Audio array processing
- deepgram-sdk>=3.5                  # Streaming ASR
- groq>=0.9                          # LLM provider

Dev dependencies (--group dev):
- pytest>=8.0
- pytest-asyncio>=0.23
- pytest-cov>=5.0
- ruff>=0.4
- mypy>=1.10
- httpx>=0.27                        # For FastAPI TestClient

Run poetry install after adding all deps.
Commit: chore(deps): add all backend Python dependencies
```

**Task 0.2: First asyncio Exercise  Agent: Cursor**

```
Create backend/app/utils/async_examples.py

Write three example functions that demonstrate asyncio understanding:

1. `run_concurrently(tasks: list[Coroutine]) -> list[Any]`
   - Runs a list of coroutines concurrently using asyncio.gather
   - Returns list of results in same order as input
   - Logs how long it took total vs sequential

2. `retry_with_backoff(coro_func: Callable, max_retries: int = 3) -> Any`
   - Retries a coroutine function with exponential backoff
   - Delays: 1s, 2s, 4s (doubles each time)
   - Adds ±25% jitter to each delay
   - Raises last exception if all retries fail
   - Logs each retry attempt with reason

3. `timeout_with_fallback(coro: Coroutine, timeout_secs: float, fallback: Any) -> A
   - Runs a coroutine with a timeout
   - Returns fallback value if timeout occurs
   - Logs a warning when fallback is used

All functions must have:
- Type hints
- Docstrings
- structlog logging

Then write tests for all three in backend/tests/unit/test_async_utils.py
Tests should verify behavior without needing real async I/O.
```

```
Commit each file separately:
feat(utils): add async utility functions with retry and timeout
test(utils): add tests for async utility functions
```

**Task 0.3: First WebSocket Exercise  Agent: Claude Code**

```
Read CONTEXT.md and /docs/doc2-system-design-architecture.md section "WebSocket Arc

Create a minimal WebSocket echo server at backend/app/api/ws_echo.py

It should:
1. Accept WebSocket connections at /ws/echo/{client_id}
2. Print "Client {client_id} connected" on connect
3. Echo back any text message prefixed with "Echo: "
4. Echo back any binary message unchanged
5. Print "Client {client_id} disconnected" on disconnect
6. Handle disconnects gracefully (WebSocketDisconnect exception)

Add a ping/pong heartbeat:
- Server sends {"type": "ping", "ts": <unix_timestamp>} every 30 seconds
- If client doesn't respond within 5 seconds, close connection
- Log a warning when a connection is closed due to missed heartbeat

Create backend/app/main.py that:
- Creates a FastAPI app
- Includes the ws_echo router
- Has GET /api/health returning {"status": "healthy"}
- Adds CORS middleware allowing localhost:5173

Add a simple test at backend/tests/integration/test_ws_echo.py:
- Test that connecting and sending "hello" returns "Echo: hello"
- Test that connecting and not sending auth closes after 5 seconds
- Use FastAPI TestClient with WebSocket support

Commits:
feat(ws): add WebSocket echo server with heartbeat monitoring
feat(api): add FastAPI main app with health endpoint and CORS
test(ws): add integration tests for WebSocket echo endpoint
```

**Task 0.4: Audio Byte-Level Exercise  Agent: Cursor**

```
Create backend/app/audio/understanding.py

This is a learning exercise — write code that demonstrates understanding of audio:

def audio_stats(audio: np.ndarray, sample_rate: int = 16000) -> dict:
    """
    Given a numpy int16 array of audio, return:
```

```
    - duration_seconds: float
    - size_bytes: int
    - size_kb: float
    - frame_count_20ms: int (how many 20ms frames)
    - max_amplitude: int
    - rms_amplitude: float (root mean square — rough loudness measure)
    - is_likely_silence: bool (rms < 500)
    """

def pcm_to_float32(audio: np.ndarray) -> np.ndarray:
    """Convert int16 PCM to float32 normalized to [-1.0, 1.0]"""

def float32_to_pcm(audio: np.ndarray) -> np.ndarray:
    """Convert float32 [-1.0, 1.0] to int16 PCM"""

def split_into_frames(audio: np.ndarray, frame_duration_ms: int = 20,
                      sample_rate: int = 16000) -> list[np.ndarray]:
    """Split audio array into fixed-size frames. Drop incomplete last frame."""

Write tests for all four functions in backend/tests/unit/test_audio_understanding.p
Use synthetic audio: np.zeros, np.ones * 1000, np.random.randint for noise.

Commit: feat(audio): add audio conversion and analysis utilities
Commit: test(audio): add unit tests for audio utility functions
```

**Phase 0 completion commit:**

```
git checkout develop
git merge feature/phase0-foundations
git tag -a v0.1.0-phase0 -m "Phase␣0␣complete:␣asyncio,␣WebSocket,␣and␣audio␣founda
git push origin develop --tags
```

**Update CONTEXT.md before Phase 1:**

```
## Current Phase: 1 — Local Voice Pipeline
## Current Task: VAD-triggered audio recording

## What's Working
- [x] Full dev environment set up
- [x] GitHub Actions CI running
- [x] WebSocket echo server
- [x] asyncio utility functions (retry, timeout, concurrent)
- [x] Audio byte-level utilities

## Key Files
- Main app: backend/app/main.py
- WebSocket: backend/app/api/ws_echo.py
- Audio utils: backend/app/audio/understanding.py
```

---

**PHASE 1: Local Voice Pipeline (Week 4-6)**

**Git branch:** feature/phase1-local-pipeline

**Task 1.1: VAD Recorder  Agent: Claude Code**

```
Read CONTEXT.md first.
Read /docs/doc2-system-design-architecture.md section "Voice Activity Detection" fu

Build the VADRecorder at backend/app/audio/vad_recorder.py

Full spec:

Class: VADRecorder
Constructor params (all with defaults):
- sample_rate: int = 16000
- frame_duration_ms: int = 20        # 20ms frames = 640 bytes at 16kHz
- silence_threshold_ms: int = 400    # Stop after 400ms silence
- min_speech_ms: int = 300           # Require 300ms minimum speech
- vad_aggressiveness: int = 2        # WebRTC VAD mode 0-3

Methods:
- record_until_silence() -> np.ndarray
  Records from microphone until sustained silence detected.
  Returns int16 numpy array of speech audio.
  Returns empty array if no speech detected (< min_speech_ms).
  Prints status: "Listening...", "Speech detected", "Silence"

- is_speech_frame(frame: bytes) -> bool
  Returns True if this 20ms frame contains speech.
  Frame must be exactly frame_duration_ms * sample_rate / 1000 * 2 bytes.

- collect_from_file(path: str) -> list[np.ndarray]
  For testing: reads a WAV file and runs VAD on it.
  Returns list of speech segments (each is a numpy array).

Logging:
- Use structlog
- Log: frame count, duration, final audio size on each recording

Error handling:
- PortAudioError: log warning "Microphone not available" and raise
- ValueError for wrong frame size: raise with clear message

Tests at backend/tests/unit/test_vad_recorder.py:
- Test is_speech_frame with synthetic silence (zeros) → should return False
- Test is_speech_frame with synthetic noise → may vary (just test no crash)
- Test collect_from_file with a silent WAV → should return empty list
```

```
- Test collect_from_file with mixed audio → test basic behavior
  (Create synthetic WAV files using soundfile in test fixtures)

Commits (separate):
feat(vad): add VADRecorder with WebRTC VAD and configurable silence threshold
test(vad): add unit tests for VADRecorder with synthetic audio fixtures
```

**Task 1.2: Faster-Whisper ASR Service  Agent: Claude Code**

```
Read CONTEXT.md first.
Read /docs/doc2-system-design-architecture.md section "ASR: How Whisper and Deepgra

Build the local ASR service at backend/app/services/asr_local.py

This is the FALLBACK ASR (offline, no API key needed).
Primary ASR (Deepgram) comes in Phase 2.

Class: LocalASR
Constructor:
- model_size: str = "small"   # tiny, small, medium
- device: str = "cpu"
- compute_type: str = "int8"
- language: str = "en"

Initialize WhisperModel ONCE in __init__ (not per transcription —
this takes 10-30 seconds).

Methods:

async def transcribe(self, audio: np.ndarray) -> TranscriptionResult | None:
    """
    Transcribes speech from audio array.
    Returns None if:
      - no_speech_prob > 0.6 (likely silence or noise)
      - compression_ratio > 2.4 (likely hallucination)
      - transcript is empty after stripping

    Runs in thread pool (asyncio.to_thread) so it doesn't block event loop.
    Logs: transcript, no_speech_prob, duration_seconds, latency_ms
    """

Pydantic model for return type:
class TranscriptionResult(BaseModel):
    text: str
    no_speech_prob: float
    language: str
    duration_seconds: float
    latency_ms: int
```

```
IMPORTANT: Use asyncio.to_thread() to run faster-whisper inference.
faster-whisper is CPU-bound and will block the event loop without this.
Example: result = await asyncio.to_thread(self._transcribe_sync, audio)

Tests at backend/tests/unit/test_asr_local.py:
Mock the WhisperModel — don't actually load it in tests (too slow).
Test:
- transcribe() returns None when no_speech_prob = 0.8
- transcribe() returns None when compression_ratio = 3.0
- transcribe() returns TranscriptionResult with correct fields on valid output
- transcribe() handles empty audio gracefully

Commits:
feat(asr): add LocalASR service with faster-whisper and hallucination prevention
test(asr): add unit tests for LocalASR with mocked WhisperModel
```

**Task 1.3: Groq LLM Service  Agent: Claude Code**

```
Read CONTEXT.md first.

Build the LLM service at backend/app/services/llm_service.py

This service handles all LLM communication with multi-provider fallback.

Pydantic models:
class Message(BaseModel):
    role: Literal["system", "user", "assistant"]
    content: str

class LLMResponse(BaseModel):
    content: str
    provider: str       # "groq", "cerebras", "gemini"
    model: str
    latency_ms: int
    tokens_used: int

Class: LLMService
Constructor: loads all providers from environment, initializes circuit breakers

Providers to implement (in fallback order):
1. Groq: GROQ_API_KEY, model="llama-3.3-70b-versatile"
2. Cerebras: CEREBRAS_API_KEY, model="llama3.3-70b" (if key present)
3. Gemini: use google-generativeai (if key present)

Methods:

async def generate(
```

```
    self,
    messages: list[Message],
    max_tokens: int = 200,
    temperature: float = 0.7
) -> LLMResponse:
    """
    Tries providers in order. Skips provider if circuit breaker is OPEN.
    Raises RuntimeError if all providers fail.
    """

async def stream(
    self,
    messages: list[Message],
    max_tokens: int = 200,
    temperature: float = 0.7
) -> AsyncIterator[str]:
    """
    Streams tokens from the fastest available provider.
    Falls back to generate() (non-streaming) if streaming fails.
    """

Circuit breaker per provider:
- Open after 3 consecutive failures
- Half-open after 60 seconds
- Close on first success in half-open state

Logging:
- Log: provider used, model, latency_ms, token count on each call
- Log warning on provider failure with error message
- Log when circuit breaker opens/closes

Tests at backend/tests/unit/test_llm_service.py:
- Mock all provider clients
- Test: Groq succeeds → uses Groq
- Test: Groq fails (RateLimitError) → falls back to Cerebras
- Test: All providers fail → raises RuntimeError
- Test: Circuit breaker opens after 3 failures
- Test: Circuit breaker allows one request in half-open state

Commits (one per logical unit):
feat(llm): add LLMService with multi-provider fallback chain
feat(llm): add CircuitBreaker for provider health management
test(llm): add unit tests for LLMService with mocked providers
```

**Task 1.4: Local Pipeline Integration (Toy)  Agent: Cursor**

Create backend/scripts/local_voice_loop.py

```
This script runs a complete local voice loop (no WebSocket, no server).
It's the Phase 1 proof-of-concept.

Flow:
1. Initialize: VADRecorder, LocalASR, LLMService, edge_tts
2. Print startup message with initialization time
3. Loop:
   a. record_until_silence() → audio
   b. If empty audio → print "No speech detected" → continue
   c. transcribe(audio) → TranscriptionResult | None
   d. If None → print "Unclear speech, try again" → continue
   e. Print transcript
   f. Add to conversation_history (last 8 messages only)
   g. generate() with interview system prompt → LLMResponse
   h. Print AI response + latencies
   i. edge_tts synthesize → play audio
   j. Print turn latency breakdown

System prompt to use:
"You are a neutral behavioral interviewer. Ask one follow-up question per turn
based on what the candidate said. Keep responses under 40 words. No bullet points."

Add --rounds flag: run N rounds then exit (useful for automated testing)
Add --model flag: choose whisper model size (default: small)

Handle KeyboardInterrupt gracefully: print session summary (turns, avg latency)

Commit: feat(pipeline): add local voice loop script for Phase 1 validation
```

**Phase 1 completion:**

```
# Run the local loop and confirm it works
cd backend && python scripts/local_voice_loop.py --rounds 3

git checkout develop
git merge feature/phase1-local-pipeline
git tag -a v0.2.0-phase1 -m "Phase 1 complete: local voice pipeline end-to-end"
git push origin develop --tags
```

---

**PHASE 2: Real-Time Streaming (Week 7-9)**

**Git branch:** feature/phase2-websocket-streaming

**Task 2.1: WebSocket Voice Handler  Agent: Claude Code**

```
Read CONTEXT.md first.
Read /docs/doc2-system-design-architecture.md sections:
```

- "WebSocket Architecture" (full section including message protocol)
- "Full Pipeline Data Flow" (the annotated timing diagram)

Build the production WebSocket voice handler at backend/app/api/ws_voice.py

Replace ws_echo.py — this is the real thing.

Message protocol: EXACTLY as specified in /docs/doc2-system-design-architecture.md "SpeakPrep WebSocket Message Protocol" section.

Handler: voice_endpoint(websocket: WebSocket, session_id: str)

State machine per connection:
IDLE → LISTENING → PROCESSING → SPEAKING → IDLE
Handle barge-in: SPEAKING → LISTENING (immediately)

Authentication:
- First message must be {"type": "auth", "token": "..."} within 5 seconds
- For Phase 2: validate token is present (real JWT validation in Phase 5)
- Close with code 4001 if missing, 4002 if timeout

Audio handling:
- Receive binary frames, append to audio_buffer: bytearray
- When audio_buffer > 3200 bytes (100ms minimum) AND state is LISTENING:
  - Check last 20ms frame (640 bytes) for silence using WebRTC VAD
  - If sustained silence (400ms) → trigger transcription
- Clear buffer after triggering

Processing pipeline (run as asyncio.Task, non-blocking):
1. ASR: transcribe audio buffer
2. If transcript empty → send {"type": "error", "message": "Unclear speech"}
3. Send {"type": "transcript", "text": ..., "is_final": true}
4. Stream LLM → sentence buffer → TTS → send binary chunks
5. Send {"type": "turn_complete", "latencies": {...}}

Barge-in detection:
- While state == SPEAKING and incoming binary frame detected:
  - Set state = LISTENING immediately
  - Cancel current TTS task if running
  - Send {"type": "barge_in_acknowledged"}
  - Clear audio buffer

ConnectionManager class:
- active_connections: dict[str, WebSocket]
- connect(), disconnect(), send_json(), send_bytes() methods

Heartbeat:
- Server sends ping every 20 seconds

```
- Close connection if no pong within 5 seconds

Error handling:
- WebSocketDisconnect: log and clean up
- Any exception in processing pipeline: send error message, continue loop

Latency tracking:
- Record timestamp at: audio_end, asr_done, llm_first_token, tts_first_chunk
- Include all deltas in turn_complete message

Commits (one per class/feature):
feat(ws): add ConnectionManager for WebSocket session tracking
feat(ws): add voice_endpoint with state machine and message protocol
feat(ws): add heartbeat monitoring with automatic dead connection cleanup
feat(ws): add barge-in detection and TTS cancellation
test(ws): add integration tests for voice WebSocket endpoint
```

**Task 2.2: Deepgram Streaming ASR  Agent: Claude Code**

```
Read CONTEXT.md first.
Read /docs/doc2-system-design-architecture.md section on Deepgram vs Whisper.

Build the streaming ASR service at backend/app/services/asr_deepgram.py

This is the PRIMARY ASR (replaces LocalASR for real-time sessions).

Class: DeepgramStreamingASR

async def transcribe_buffer(
    self,
    audio_buffer: bytes,
    sample_rate: int = 16000
) -> TranscriptionResult | None:
    """
    Sends complete audio buffer to Deepgram REST API.
    Used when we have a complete utterance (post-VAD).
    Returns None if transcript is empty.
    """

async def stream_to_deepgram(
    self,
    audio_queue: asyncio.Queue[bytes | None],
    callback: Callable[[str, bool], Awaitable[None]]
) -> None:
    """
    Opens a streaming connection to Deepgram.
    Reads audio chunks from queue (None = end of stream).
    Calls callback(transcript, is_final) for each result.
```

```
    is_final=True when Deepgram signals end of utterance.
    """

Configuration:
- model="nova-3", language="en"
- encoding="linear16", sample_rate=16000, channels=1
- smart_format=True, interim_results=True
- endpointing=380 (380ms silence detection)

Fallback: if Deepgram fails (any exception), fall back to LocalASR.transcribe()
Log which provider was used for each transcription.

Wrap DeepgramStreamingASR + LocalASR in a single facade:

Class: ASRService
async def transcribe(self, audio: bytes | np.ndarray) -> TranscriptionResult | None
    Tries Deepgram first, LocalASR as fallback. Logs provider used.

Tests:
- Mock Deepgram SDK
- Test: Deepgram returns result → ASRService returns it
- Test: Deepgram raises exception → falls back to LocalASR
- Test: Both fail → returns None

Commits:
feat(asr): add DeepgramStreamingASR with streaming and batch transcription
feat(asr): add ASRService facade with →DeepgramLocalASR fallback
test(asr): add tests for ASRService fallback logic
```

**Task 2.3: Kokoro TTS Service  Agent: Claude Code**

```
Read CONTEXT.md first.

Build the TTS service at backend/app/services/tts_service.py

Kokoro TTS runs locally at http://kokoro:8880 (Docker) or http://localhost:8880 (de

Class: KokoroTTS

async def synthesize(self, text: str, voice: str = "af_heart") -> bytes:
    """Returns complete audio as bytes (PCM format)."""

async def synthesize_stream(
    self,
    text: str,
    voice: str = "af_heart"
) -> AsyncIterator[bytes]:
    """Streams audio chunks as they're generated."""
```

```
async def synthesize_sentence_pipeline(
    self,
    text_stream: AsyncIterator[str],
    websocket: WebSocket,
    session_id: str
) -> None:
    """
    Takes a stream of text tokens from LLM.
    Accumulates into sentences using split_sentences().
    For each complete sentence: synthesize_stream() → send binary chunks to WebSoc
    Prepends 4-byte sequence number to each chunk.
    Runs TTS for sentence N+1 concurrently with playback of sentence N.
    """

def split_sentences(self, text: str) -> tuple[list[str], str]:
    """
    Splits text on sentence boundaries: ". " "! " "? " ";\n"
    Returns: (complete_sentences, remainder)
    Handles abbreviations: Dr. Mr. Mrs. Ms. Prof. Sr. Jr. vs. etc. Inc. e.g. i.e.
    Handles numbers: 3.14, $4.5M (not sentence boundaries)
    """

Available voices to expose as constants:
VOICE_WARM_FEMALE = "af_heart"
VOICE_NEUTRAL_MALE = "am_adam"
VOICE_BRITISH_FEMALE = "bf_emma"
VOICE_BRITISH_MALE = "bm_george"

Error handling:
- If Kokoro unreachable: log error, raise TTSError
- If text is empty: return empty bytes silently

Tests (focus on split_sentences — unit testable):
test_split_sentences:
- "Hello. World." → ["Hello.", " World."], ""
- "Dr. Smith said hi." → [], "Dr. Smith said hi."
- "Go to speakprep.com now!" → ["Go to speakprep.com now!"], ""
- "I earned $3.5M. Not bad!" → ["I earned $3.5M.", " Not bad!"], ""
- "Hi! How are you?" → ["Hi!", " How are you?"], ""
At minimum 10 test cases covering edge cases.

Mock httpx for synthesize() tests.

Commits:
feat(tts): add KokoroTTS service with streaming synthesis
feat(tts): add sentence splitter with abbreviation handling
test(tts): add 10+ unit tests for sentence boundary detection
```

```
test(tts): add unit tests for KokoroTTS with mocked httpx
```

**Task 2.4: Simple Browser Client  Agent: Antigravity or manual**

```
Create a minimal test client at frontend/index.html (single file, no framework yet)

It needs:
1. Connect button: connects WebSocket to ws://localhost:8000/ws/voice/{uuid}
   - On open: send auth token {"type": "auth", "token": "dev-token"}

2. Push-to-talk button (hold to record):
   - mousedown/touchstart: start Web Audio API capture at 16kHz mono
   - mouseup/touchend: stop capture, send {"type": "audio_end"}
   - While held: send raw Int16 PCM via WebSocket.send(buffer)
   - Show recording indicator (red dot) while held

3. Display panel:
   - Transcript: show text from {"type": "transcript"} messages
   - AI response: accumulate tokens from {"type": "llm_token"}
   - Latency: show total_ms from {"type": "turn_complete"}

4. Audio playback:
   - Receive binary WebSocket frames (4-byte seq number + PCM audio)
   - Sort by sequence number
   - Decode PCM Float32 at 24kHz (Kokoro output format)
   - Play in order using Web Audio API

5. Error display: show {"type": "error"} messages in red

No framework. Pure HTML/JS. Must work in Chrome.

Commit: feat(frontend): add minimal single-file test client for WebSocket voice
```

**Phase 2 completion:**

```
# Test manually: open frontend/index.html, connect, speak, hear response
# Measure: what is the actual end-to-end latency? Update CONTEXT.md.

git checkout develop
git merge feature/phase2-websocket-streaming
git tag -a v0.3.0-phase2 -m "Phase 2 complete: real-time WebSocket voice streaming"
git push origin develop --tags
```

---

**PHASE 3: Interview Intelligence (Week 10-12)**

**Git branch:** feature/phase3-interview-intelligence

**Task 3.1: Database Setup  Agent: Claude Code**

```
Read CONTEXT.md and /docs/doc2-system-design-architecture.md section "Database Desi

Set up the database layer.

1. Create backend/app/models/schema.py with SQLAlchemy models for:
   - User, Resume, Session, Turn, Question, UserQuestionHistory, UserEloRating
   EXACTLY as specified in the schema in doc2 (copy the SQL, convert to SQLAlchemy)
   Use SQLAlchemy 2.0 async style with mapped_column() and Mapped[].

2. Create backend/app/database.py:
   - Async engine using asyncpg: create_async_engine(DATABASE_URL)
   - AsyncSession factory: async_session = async_sessionmaker(engine)
   - async def get_db() -> AsyncIterator[AsyncSession]: (FastAPI dependency)
   - async def init_db(): creates all tables (for testing only)

3. Set up Alembic:
   cd backend && alembic init alembic
   Edit alembic/env.py to use async engine and import Base from models/schema.py
   alembic revision --autogenerate -m "initial schema"
   alembic upgrade head   (run against test DB)

4. Create backend/app/repositories/ (one file per model):
   - user_repo.py: get_by_id, get_by_email, create, update
   - session_repo.py: create, get_by_user, update_scores
   - turn_repo.py: create, get_by_session
   - question_repo.py: get_candidates_for_user, get_by_id
   Each repo takes AsyncSession, uses SQLAlchemy 2.0 select() syntax.

Tests at backend/tests/integration/test_repositories.py:
Use pytest-asyncio with a real test database (from CI services).
Test basic CRUD for User and Session repos.

Commits (one per file):
feat(db): add SQLAlchemy models matching spec schema
feat(db): add async database engine and session factory
feat(db): add Alembic migration setup and initial schema migration
feat(db): add User and Resume repositories with CRUD operations
feat(db): add Session and Turn repositories
feat(db): add Question repository with ELO-based candidate selection
test(db): add integration tests for repository CRUD operations
```

**Task 3.2: Resume Parser  Agent: Claude Code**

```
Read CONTEXT.md first.

Build backend/app/services/resume_parser.py
```

```
async def parse_resume_pdf(pdf_bytes: bytes) -> ResumeData:
    """
    Extracts structured data from a resume PDF.

    Step 1: PyMuPDF text extraction
      - Use pymupdf (pip name: PyMuPDF) to extract text
      - If text < 100 characters: raise ValueError("PDF appears image-based")

    Step 2: LLM structured extraction
      - Send extracted text to Groq Llama 70B
      - Prompt instructs model to return ONLY valid JSON (no markdown)
      - Temperature=0 for deterministic output
      - Parse JSON response into ResumeData Pydantic model

    Step 3: Post-processing
      - Calculate experience_years from work experience dates
      - Infer skill_level: junior (<2yr), mid (2-5yr), senior (5-10yr), staff (10+)
      - Extract top 5 most impressive highlights (for interview context)
      - Return ResumeData

Error handling:
- JSONDecodeError: retry once with more explicit prompt
- PyMuPDF error: raise ParseError("Could not read PDF")
- ValidationError: raise ParseError with field that failed

class ResumeData(BaseModel):
    name: str
    email: str | None
    work_experience: list[WorkExperience]
    education: list[EducationEntry]
    skills: list[str]
    projects: list[ProjectEntry]
    years_of_experience: float
    skill_level: Literal["junior", "mid", "senior", "staff"]
    top_highlights: list[str]   # For interview personalization
    raw_text: str               # Store original for re-parsing

Add dependency: poetry add pymupdf

Tests:
- Create a simple fake resume as plain text and test the Pydantic parsing logic
- Mock the Groq call to return sample JSON
- Test error handling: empty PDF, malformed JSON from LLM

Commits:
feat(resume): add PDF text extraction with PyMuPDF
feat(resume): add LLM-based structured resume parsing with ResumeData model
```

```
test(resume): add unit tests for resume parser with mocked LLM
```

**Task 3.3: Question Bank + ELO Selector  Agent: Claude Code**

```
Read CONTEXT.md first.

Build backend/app/services/question_selector.py

class QuestionSelector:
    TARGET_SUCCESS_RATE: float = 0.67

    async def select_next_question(
        self,
        db: AsyncSession,
        user_id: str,
        category: str,   # "behavioral" | "technical" | "system_design"
        resume_data: ResumeData | None = None,
        exclude_ids: list[str] = []
    ) -> Question:
        """
        Selects the question where user's expected success ≈ 0.67.
        Formula: ideal question ELO ≈ user_elo − 170 (for 67% expected success).
        Adds randomness: weighted random from top 5 candidates.
        Excludes recently asked questions (exclude_ids).
        Falls back to any active question if no candidates found.
        If resume_data provided: prefer questions matching user's experience.
        """

    def update_elo_ratings(
        self,
        user_elo: float,
        question_elo: float,
        normalized_score: float,  # -0.01.0 (score/5.0)
        k_factor: int = 32
    ) -> tuple[float, float]:
        """
        Returns (new_user_elo, new_question_elo).
        Clamps both to range [600, 2400].
        """

    async def record_interaction(
        self,
        db: AsyncSession,
        user_id: str,
        question_id: str,
        turn_id: str,
        score: float   # -1.05.0
    ) -> None:
```

```
        """
        Updates user ELO and question ELO in database.
        Records in user_question_history.
        """
```

```
Build backend/app/data/seed_questions.py:
A script that inserts 50 starter questions into the database.
Include at minimum:
- 20 behavioral (leadership, conflict, failure, teamwork, motivation)
- 15 technical (algorithms, data structures, system concepts)
- 15 system design (URL shortener, rate limiter, news feed, chat system)
Each with: text, category, subcategory, difficulty 1-5, tags, elo_rating=1200

Commits:
feat(questions): add QuestionSelector with ELO-based adaptive selection
feat(questions): add ELO rating update and history tracking
feat(questions): add seed script with 50 starter questions across all categories
test(questions): add unit tests for ELO calculation and selector logic
```

**Task 3.4: Interview AI System Prompts  Agent: Cursor**

```
Create backend/app/services/interview_prompts.py

This file contains all system prompts for the AI interviewer.
No code logic here — just prompts as constants.

def get_behavioral_prompt(
    candidate_name: str,
    resume_summary: str,
    target_role: str,
    target_company: str,
    persona: Literal["friendly", "neutral", "challenging", "adversarial"],
    remaining_minutes: int,
    questions_asked: int
) -> str:
    """Returns the behavioral interview system prompt."""

def get_technical_prompt(...same params...) -> str:
    """Returns the technical interview system prompt."""

def get_system_design_prompt(...same params...) -> str:
    """Returns the system design interview system prompt."""

def get_scoring_prompt(
    question_text: str,
    candidate_transcript: str,
    question_type: str
) -> str:
```

```
    """Returns the scoring prompt for post-turn evaluation."""

Rules for ALL voice interview prompts:
1. Responses must be under 40 words (enforced in prompt)
2. No bullet points, no markdown, no numbered lists
3. One question per turn only
4. Use natural fillers: "I see.", "Go on.", "Interesting."
5. Probe for STAR components when missing (specific probes for each)
6. Persona affects tone:
   - friendly: "Great point! Can you tell me more about..."
   - neutral: "Tell me more about the action you took."
   - challenging: "That sounds like what the team did. What did YOU do?"
   - adversarial: "I'm not convinced. Give me a more specific example."

The scoring prompt must:
- Return ONLY valid JSON (no markdown wrapping)
- Include all 5 score dimensions with rubrics
- Include strengths, improvements, example_better_answer
- Include STAR component boolean analysis

Write one test that calls get_behavioral_prompt() and verifies
the output contains key phrases ("40 words", "one question", persona phrase).

Commits:
feat(prompts): add interview system prompts for behavioral, technical, system desig
feat(prompts): add scoring prompt with 5-dimension rubric and STAR analysis
test(prompts): add basic tests for prompt generation
```

**Task 3.5: Scoring Service  Agent: Claude Code**

```
Read CONTEXT.md first.

Build backend/app/services/scoring_service.py

class ScoringService:

    async def score_turn(
        self,
        transcript: str,
        question: Question,
        session_context: dict
    ) -> TurnScores:
        """
        Scores a single interview turn.
        Runs ASYNC — does not block the voice session.
        Uses get_scoring_prompt() from interview_prompts.py.
        Calls LLM with temperature=0 (deterministic).
        Parses JSON response into TurnScores.
```

```
        Retries once if JSON parsing fails.
        """

    def compute_filler_word_stats(self, transcript: str) -> FillerStats:
        """
        Counts filler words: um, uh, like, you know, basically, literally,
        actually, sort of, kind of, right? (at end of sentence)
        Returns: FillerStats(total_count, rate_per_minute, filler_list)
        Requires word count and duration for rate calculation.
        """

    def estimate_wpm(self, transcript: str, duration_seconds: float) -> float:
        """Words per minute from transcript and recorded duration."""

    async def generate_session_summary(
        self,
        session_id: str,
        turns: list[Turn],
        db: AsyncSession
    ) -> SessionSummary:
        """
        After session completes, generates overall summary.
        Calls LLM with all turn transcripts and scores.
        Returns: overall_score, top_strengths, top_improvements, trends.
        Saves to sessions table.
        """

class TurnScores(BaseModel):
    content_score: float
    communication_score: float
    star_score: float
    confidence_score: float
    filler_score: float
    overall_score: float
    strengths: list[str]
    improvements: list[str]
    example_better_answer: str
    has_situation: bool
    has_task: bool
    has_action: bool
    has_result: bool
    filler_stats: FillerStats
    wpm: float

Weights for overall_score:
content=0.30, communication=0.25, star=0.20, confidence=0.15, filler=0.10

Tests (focus on pure functions):
```

```
test compute_filler_word_stats:
- "um I think, like, you know, it was sort of good" → count=5
- "I led the team and delivered on time." → count=0
- Empty string → count=0

test estimate_wpm:
- 100 words in 60 seconds → 100 WPM
- 140 words in 60 seconds → 140 WPM

Mock LLM for score_turn tests.

Commits:
feat(scoring): add ScoringService with 5-dimension LLM-based scoring
feat(scoring): add filler word detection and WPM estimation
test(scoring): add unit tests for filler detection and WPM with known inputs
test(scoring): add unit tests for TurnScores JSON parsing
```

**Phase 3 completion:**

```
git checkout develop
git merge feature/phase3-interview-intelligence
git tag -a v0.4.0-phase3 -m "Phase␣3␣complete:␣resume␣parsing,␣scoring,␣ELO␣questio
git push origin develop --tags
```

---

**PHASE 4: Deploy to Production (Week 13-14)**

**Git branch:** feature/phase4-deployment

**Task 4.1: Docker Compose Production Config  Agent: Claude Code**

```
Create docker-compose.prod.yml at the project root.

Copy the exact Docker Compose config from /docs/doc2-system-design-architecture.md
section "Docker Compose (Production)" and adapt it:

Services needed:
- caddy: reverse proxy with HTTPS
- app: our FastAPI backend (built from ./backend/Dockerfile)
- whisper: faster-whisper-server (ARM-compatible image)
- kokoro: kokoro-fastapi-cpu (ARM-compatible image)
- valkey: valkey/valkey:8-alpine with persistence

Networks:
- frontend: caddy + app
- backend (internal: true): app + whisper + kokoro + valkey

Create backend/Dockerfile:
```

```
- Multi-stage: builder (install deps) + production (minimal image)
- Base: python:3.12-slim
- Copy only what's needed (not .git, not tests)
- Run as non-root user (appuser)
- Healthcheck: curl -f http://localhost:8000/api/health || exit 1
- CMD: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1

Create Caddyfile:
EXACTLY as specified in doc2 section "Caddyfile".
Your domain: whatever you registered.
Include flush_interval -1 for WebSocket.

Create .dockerignore:
__pycache__/, .venv/, .env, *.pyc, .git/, tests/, docs/

Test locally:
docker compose -f docker-compose.prod.yml build app
docker compose -f docker-compose.prod.yml up app valkey
curl http://localhost:8000/api/health

Commits:
feat(infra): add multi-stage Dockerfile for FastAPI backend
feat(infra): add docker-compose.prod.yml with all services
feat(infra): add Caddyfile with HTTPS and WebSocket support
chore(infra): add .dockerignore to exclude dev files from image
```

**Task 4.2: Oracle ARM Provisioning  Agent: Yourself (manual — no agent)**

This step cannot be automated. Follow these steps manually:

```
# 1. Go to oracle.com/cloud/free → Sign up
# 2. Create VM: Compute → Instances → Create
#    Shape: VM.Standard.A1.Flex
#    OCPU: 4, RAM: 24 GB
#    Image: Canonical Ubuntu 22.04
#    Add your SSH public key
#    VCN: Create new, add security rule: TCP 22/80/443 from 0.0.0.0/0
# 3. Wait ~5 minutes for provisioning

# 4. SSH in
ssh ubuntu@YOUR_ORACLE_IP

# 5. Initial server setup (run on server)
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker ubuntu
newgrp docker  # or log out and back in
sudo apt install docker-compose-plugin git -y
```

```
# 6. Cloudflare Tunnel setup (on server)
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudfla
  -o /usr/local/bin/cloudflared
chmod +x /usr/local/bin/cloudflared
cloudflared login
cloudflared tunnel create speakprep
cloudflared tunnel route dns speakprep api.yourdomain.com
sudo cloudflared service install
sudo systemctl enable cloudflared && sudo systemctl start cloudflared

# 7. Add GitHub Actions secrets
# github.com/abhiyansainju/speakprep → Settings → Secrets → Actions:
# ORACLE_HOST = YOUR_ORACLE_IP
# ORACLE_SSH_KEY = (contents of ~/.ssh/id_rsa private key)

# 8. Commit this task as documentation

git commit --allow-empty -m "docs(infra): Oracle ARM VM provisioned and Cloudflare

VM: VM.Standard.A1.Flex, 4 OCPU, 24 GB RAM, Ubuntu 22.04
Region: us-ashburn-1
Tunnel: speakprep → api.yourdomain.com
GitHub secrets: ORACLE_HOST, ORACLE_SSH_KEY configured"
```

**Phase 4 completion:**

```
# First real deploy:
git checkout main
git merge develop
git push origin main
# Watch GitHub Actions — deploy workflow should run

# Verify:
curl https://api.yourdomain.com/api/health

git tag -a v0.5.0-phase4 -m "Phase 4 complete: deployed to Oracle ARM with HTTPS"
git push origin main --tags
```

---

**PHASE 5: UI Mockups + Frontend (Week 15–20)**

**Before writing any React code — do mockups first.**

**Task 5.0: Figma Mockups (1 week before Phase 5 code) Agent: Yourself + Antigravity for research**

**Antigravity research prompt:**

```
Go to these competitor apps and screenshot the main voice/practice interface:
1. yoodli.ai (their practice session screen)
2. Google Interview Warmup (warmup.withgoogle.com)
3. Huru app (if accessible)

For each, screenshot:
- The main session screen (what you see while practicing)
- The results/feedback screen after a session
- The dashboard/history screen

Do not sign up or enter personal information.
Report: what elements are on each screen, what's missing, what could be better.
```

**After research, build Figma mockups for:** 1. **Voice Session screen** (most important): microphone button, live transcript pane, AI response text, latency indicator, turn counter, timer, filler word counter 2. **Post-Session Report** screen: overall score ring, 5 dimension bars, per-question expandable cards 3. **Progress Dashboard**: line chart (score trend), topic coverage grid, streak counter

Only mock these 3 screens. Everything else (landing, auth, setup) can be wire-framed quickly or built directly in code.

**Task 5.1: React Frontend Agent: Claude Code (with Cursor for components)**

```
Initialize the React frontend.

cd frontend
npm create vite@latest . -- --template react-ts
npm install

Dependencies to add:
npm install @supabase/supabase-js react-router-dom zustand
npm install recharts        # Progress charts
npm install react-hook-form zod   # Forms
npm install class-variance-authority clsx tailwindcss

Set up Tailwind:
npx tailwindcss init -p

Based on the Figma mockups, build these components:
(Do one component per commit — never bundle them)

1. VoiceButton (components/VoiceButton.tsx)
   - Push-and-hold interface
   - Visual waveform animation while recording
   - Keyboard shortcut: Space bar

2. TranscriptPane (components/TranscriptPane.tsx)
   - Shows live transcript as user speaks
```

```
   - Shows AI response tokens as they arrive
   - Auto-scrolls to bottom

3. SessionMetrics (components/SessionMetrics.tsx)
   - Live filler word counter
   - Speaking pace indicator (WPM)
   - Turn timer
   - Session progress

4. ScoreCard (components/ScoreCard.tsx)
   - 5 dimension bars with scores
   - Color-coded: green/yellow/red by score
   - Strengths and improvements lists

5. ProgressChart (components/ProgressChart.tsx)
   - Recharts LineChart
   - Multiple lines: overall, content, star, communication
   - Last 30 sessions

6. VoiceSessionPage (pages/VoiceSessionPage.tsx)
   - Assembles all components
   - Manages WebSocket via useVoiceSession hook
   - Handles audio recording via useAudioRecorder hook

7. hooks/useVoiceSession.ts
   - Manages WebSocket lifecycle
   - Reconnection with exponential backoff
   - State machine: idle/listening/processing/speaking
   - Emits events: onTranscript, onTokens, onTurnComplete, onError

8. hooks/useAudioRecorder.ts
   - Web Audio API: AudioContext, ScriptProcessor
   - 16kHz, mono, Int16 PCM
   - Sends chunks to WebSocket

Commits (one per component):
feat(frontend): add VoiceButton with push-to-talk and waveform
feat(frontend): add TranscriptPane with auto-scroll
[etc.]
```

---

**PHASE 6: Hardening + Launch (Week 21-24)**

**Agent: Claude Code for security and testing, Codex for scripts**

**Claude Code security prompt:**

```
Run a security review of the SpeakPrep backend.
```

```
Check for:
1. SQL injection: verify all queries use parameterized statements (SQLAlchemy alway
2. JWT implementation: check token expiry, rotation, revocation list in Valkey
3. WebSocket origin validation: only allow our domains
4. Rate limiting: verify middleware is working on all endpoints
5. Secret exposure: scan for any hardcoded strings that look like API keys
6. CORS: verify only our frontend domains are allowed
7. Input validation: all user inputs go through Pydantic models?
8. File upload: resume PDF upload — max size enforced? File type validated?

For each issue found:
- Describe the vulnerability
- Show the code where it exists
- Provide the fix
- Add a test that would catch this in future

Commit each fix separately: fix(security): [specific issue]
```

**Codex load test script prompt:**

```
Write a Locust load test at scripts/load_test.py

Test realistic user behavior:
- 30% of users: just view dashboard (GET /api/sessions)
- 50% of users: start a session (POST /api/sessions)
- 20% of users: hold a WebSocket connection for 2 minutes (simulate voice session)

Target: 50 concurrent users with < 2 second P95 latency.
Report: P50, P95, P99 latencies per endpoint.

Run with: locust -f scripts/load_test.py --host=https://api.yourdomain.com
```

**Launch PR checklist commit:**

```
git commit --allow-empty -m "chore: launch readiness checklist complete

Pre-launch verified:
 OWASP ZAP scan: no high severity issues
 Gitleaks: no secrets in git history
 Load test: 50 concurrent users, P95 < 2s
 All unit tests passing (coverage > 70%)
 All integration tests passing
 DB migrations clean (alembic history)
 UptimeRobot monitoring active
 Sentry error tracking active
 PostHog analytics active
 HTTPS everywhere (verified)
 Health check endpoint returns 200"
```

---

## PART 4: DAILY ROUTINE

Every working day, 5 minutes at the start, 5 minutes at the end.

### Morning (5 min)

```
git checkout develop
git pull origin develop

# Check what you were working on
cat CONTEXT.md | head -20

# Start feature branch
git checkout -b feature/[what-im-doing-today]

# Open the right agent with the right prompt:
# - Small edits in existing file → Cursor
# - New module or complex feature → Claude Code
# - Research → Antigravity
```

### During (every 30–60 min)

```
# Commit what you just finished
git add -p
git commit -m "feat/fix/test(scope): what you did"
```

### End of Day (5 min)

```
# Commit any remaining work
git add -p
git commit -m "wip: [description] — EOD checkpoint"

# Update CONTEXT.md
# - What did I complete today?
# - What's in progress?
# - What's blocked?
git add CONTEXT.md
git commit -m "docs: update CONTEXT.md - EOD [date]"

git push origin [current-branch]
```

### Weekly (30 min, every Friday)

```
# Merge to develop if feature is complete
git checkout develop
```

```
git merge feature/[feature-name]
git push origin develop

# Check CI is green
gh run list --limit 5

# Review: are you on pace with the phase timeline?
# Open New Relic: check P95 latency trend
# Open Sentry: any new error types?
# Open PostHog: session completion rate?
# Update timeline estimate in CONTEXT.md
```

---

*End of Document 5 — Full Development Playbook*